ohqrl User's Manual

Reinforcement-learning control of OpenHydroQual models

OpenHydroQual

November 2, 2026

Contents

1 What ohqrl does **3**
1.1 Key ideas . . . 3
1.2 The workflow . . . 4

2 Installation **4**
2.1 Requirements . . . 4
2.2 Build the library and the Python bindings . . . 4
2.3 Python environment . . . 4
2.4 Where ohqrl finds OHQ . . . 4
2.5 Check the installation . . . 5

3 Preparing the OHQ model **5**
3.1 The script . . . 5
3.2 Time units . . . 5
3.3 Controls . . . 5
3.4 Objective functions . . . 6

4 The YAML file **6**
4.1 Complete example . . . 6
4.2 Reference . . . 7

5 Running **8**
5.1 Step 1: baselines . . . 8
5.2 Step 2: training . . . 9
5.3 Step 3: choose a checkpoint . . . 9
5.4 Step 4: test . . . 9
5.5 Warm start . . . 10

6 Worked example: stormwater pond **10**

7 Using the environment from Python **10**
7.1 The OHQ stepping API . . . 11

8 Practical advice **11**
8.1 Speed 11
8.2 Long runs 12
8.3 Getting a policy that is really better 12
8.4 Troubleshooting 12
8.5 Limitations 13

1 What ohqrl does

`ohqrl` finds a *control policy* for a system modeled in OpenHydroQual (OHQ). A policy is a rule that sets controllable quantities from what is known at the time. Examples:

- a valve opening, a gate setting or a pump rate;
- a reservoir release.

The rule takes as input the current state of the system and, optionally, forecasts and the season. It is learned by *reinforcement learning* (RL): the policy is tried in the OHQ model over many simulated years, and improved step by step so that it lowers the model's own objective functions.

You provide two files:

1. **The OHQ model**, as an ordinary `.ohq` script. It must contain the objective functions to minimize.
2. **A YAML file** that names
   - the *controls*: which model properties the policy sets;
   - the *observations*: which state variables and forcings the policy sees;
   - the *objectives*: which objective functions it minimizes;
   - the time windows used for training, validation and testing.

No change to the OHQ model format is needed.

1.1 Key ideas

Control interval Δt.
The policy acts every Δt (for example every hour). The control is held constant within the interval, while the OHQ solver takes its own, smaller time steps.

Episode.
One simulated window of length T (for example 3 years), starting at a random time in the training period. Training plays many episodes.

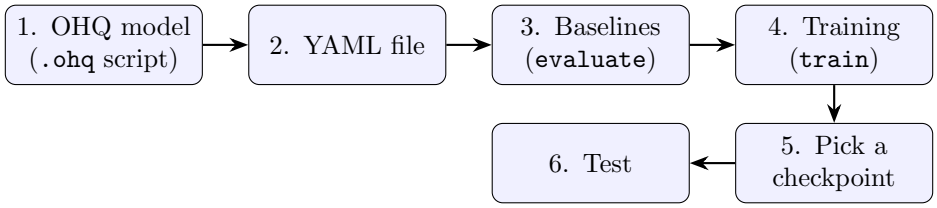
Reward.
After each interval the policy receives a reward. Here the reward is minus the weighted amount by which the objective functions grew during that interval:

$$r_k \;=\; -\sum_i \frac{w_i\,\omega_i}{s_i}\,\big(I_i(t_{k+1}) - I_i(t_k)\big),$$

where $I_i(t)$ is the integral of objective function i from the start of the episode to t, ω_i its OHQ weight, and w_i, s_i an extra weight and scale from the YAML file. Summed over an episode, the rewards are *exactly* minus the weighted objective that OHQ computes for that window, so the policy is trained on precisely what it is judged by.

Additive objectives only.
Every objective function must use the method `integrate`. Other kinds (maximum, exceedance, ...) must first be written as an integral; Section 3.4 shows how.

1.2 The workflow



Always evaluate simple fixed controls (step 3) before training. They tell you how good a policy must be to be worth having. Often a well-chosen fixed setting is already hard to beat.

2 Installation

2.1 Requirements

- OpenHydroQual source tree (this repository), with `OHQLib` and `PythonBindings` built. The bindings must include the stepping API (commit `c9b9d2b` or later).
- Python 3.10 or newer, with `numpy`, `torch`, `gymnasium` and `pyyaml` (`ohqrl/requirements.txt`).
- A multi-core CPU. Each training worker runs one OHQ model on one core.

2.2 Build the library and the Python bindings

With Qt and the OHQ dependencies installed (as for the OHQ GUI), build `OHQLib` first and then the bindings:

```
cd OHQLib && mkdir -p build/Release && cd build/Release
qmake ../../OHQLib.pro CONFIG+=release && make -j8
cd ../../../PythonBindings && mkdir -p build/Release && cd build/Release
qmake ../../PythonBindings.pro CONFIG+=release && make -j8
```

`PythonBindings.pro` sets the include paths of `pybind11` and Python, and the location of `OHQLib`. Adjust them for your system and build folder names. The build produces `openhydroqual_py*.so` (Linux, macOS) or `.pyd` (Windows).

2.3 Python environment

```
python -m venv ~/ohq_env && source ~/ohq_env/bin/activate
pip install -r ohqrl/requirements.txt
```

2.4 Where ohqrl finds OHQ

Run all commands from the OpenHydroQual folder, so that `python -m ohqrl...` finds the package. By default `ohqrl` looks for:

- the bindings under `PythonBindings/build/*/`, preferring a folder whose name contains `Release`;
- the templates in `resources/`.

Two environment variables override these defaults:

`OHQ_PY_PATH`	folder that contains `openhydroqual_py*.so`/`.pyd`
`OHQ_RESOURCES`	folder with the OHQ JSON templates (`main_components.json`, ...)

2.5 Check the installation

```
python -m ohqrl.tests.test_stepping
```

The test runs a small reservoir twice. The first time it uses one call to `solve()`. The second time it steps interval by interval and rewrites the control before each interval. It prints matching storages and objective values, and `OK`.

3 Preparing the OHQ model

3.1 The script

The model is an OHQ script (the format written by *File* → *Save as script* in the GUI, or by hand). Two placeholders are replaced before the script is loaded, so that the model folder can be moved:

`${OHQ_RESOURCES}`	the OHQ template folder
`${CONFIG_DIR}`	the folder of the YAML file (for forcing files next to it)

```
loadtemplate; filename = ${OHQ_RESOURCES}main_components.json
addtemplate; filename = ${OHQ_RESOURCES}Pond_Plugin.json
create block;type=Pond,Storage=0,alpha=250,beta=2,inflow=${CONFIG_DIR}/inflow.txt,name=
    Pond
```

The simulation start and end times in the script are irrelevant: each episode sets its own. Forcing time series (such as `inflow`) should cover the whole period that episodes are drawn from.

3.2 Time units

`ohqrl` uses the model's time unit throughout, usually days. All times in the YAML file (control interval, episode length, windows) are in that unit. The pond example counts days since 1900-01-01, so 1 hour is $1/24 = 0.0416667$.

3.3 Controls

A control is a property of a block or link that the policy sets. There are two kinds:

`value`
: An ordinary numeric property, for example the `opening` of a `controlled_orifice`, or the `Q_max` of a `Reservoir_link_rule`. It is set with `set_value` before each interval.

`timeseries`
: A time-series property, for example the `flow` of a `User_flow` link or a pump's `coefficient_time_series`. Before each interval it is replaced by a constant series holding the chosen value.

The property must be defined in the script (any initial value) and must actually drive the flow. A property that is only read at initialization does not respond to changes during the run.

3.4 Objective functions

Objective functions are created in the script. The `type` must be the template name `Objective_Function`; otherwise the script fails to load. The `method` must be `integrate`:

```
create objectivefunction;type=Objective_Function,name=Flood,object=outlet_pipe,expression
    =(_pos(flow-1000)*_pos(flow-1000)/1000),method=integrate,weight=1
create objectivefunction;type=Objective_Function,name=Young,object=outlet_pipe,expression
    =(flow*Age:concentration.s),method=integrate,weight=1
```

The expression is evaluated on the named block or link at every solver step, and its time integral is the objective. On a link, `.s` and `.e` refer to the quantities of its start and end blocks.

Writing an objective as an integral. Many goals are naturally additive. Others can be made so:

Goal	Expression (integrated)	Comment
spilled or overflow volume	`flow` (on the spill link)	exact
pollutant load	`flow*C.s`	exact
time above a threshold	`_pos(depth-h)/(depth-h+1e-9)`	smooth-ish indicator
peak flow / flood exposure	`_pos(flow-Q)*_pos(flow-Q)/Q`	penalizes large exceedances most
exceedance flow at probability p	as above, with Q near the target exceedance flow	surrogate
target storage or level	`(Storage-S)*(Storage-S)/S`	tracking
supply shortfall	`_pos(demand-supply)`	demand as a time series

Use squared excesses when large exceedances matter more than many small ones.

Weights and scales. Objectives often differ by orders of magnitude. Give each a `scale` in the YAML file, for example its value per year under a reference control, so that the weighted sum adds comparable numbers. `ohqrl.evaluate --normalize` prints each objective divided by the baseline's, which helps to choose scales.

4 The YAML file

4.1 Complete example

```
model:
  script: pond.ohq # relative to this file
  settings: # extra OHQ system settings
    initial_time_step: 0.0034722 # 5 min
    max_timestep_increase_factor: 1
time:
  control_interval: 0.0416666667 # 1 h (model time unit: days)
  episode_length: 1095 # 3 years
  train: [[29219, 43829]] # 1980-2019
  validation_starts: [35209, 40104]
  test_starts: [43829, 44925] # 2020-2022, 2023-2025
controls:
  - {name: valve, object: valve, property: opening, kind: value, min: 0, max: 1, scale:
      square}
```

```
observations:
  - {object: Pond, quantity: Storage, scale: 1000}
  - {object: Pond, quantity: "Age:concentration"}
  - {forcing: inflow.txt, windows: [[-1, 0], [0, 1], [0, 6], [0, 24]],
     scale: 1000, transform: log1p, noise: 0.3}
  - {season: 365.25}
objectives:
  - {name: Flood, weight: 1, scale: 12684}
  - {name: Young, weight: 1, scale: 18982}
snap:
  - {object: Pond, quantity: Storage, below: 0.1}
baseline: {valve: 1.0}
solver_failure_penalty: 10
training:
  total_steps: 3000000
  workers: 8
  warm_start: {controls: {valve: 0.1}}
```

4.2 Reference

Key	Default	Meaning
model		
`script`	required	OHQ script, relative to the YAML file
`settings`	{}	system settings applied after loading (any OHQ `setvalue; object=system` quantity)
time		(model time units)
`control_interval`	required	Δt: how often the policy acts
`episode_length`	required	T: length of one episode
`train`	required	list of `[start, end]` ranges; episodes start uniformly at random in them, on multiples of Δt, and must fit inside
`validation_starts`	[]	start times of fixed windows for choosing among checkpoints (inside the training period)
`test_starts`	[]	start times of fixed windows for the final comparison (outside the training period)
controls (list; one action per entry)		
`name`	object.property	label used in `baseline`, `warm_start` and `--constant`
`object`, `property`	required	block or link name, and the property set
`kind`	`value`	`value` or `timeseries`
`min`, `max`	required	range of the control
`scale`	`linear`	how the action $a \in [-1, 1]$ maps to the range, with $x = (a+1)/2$: `linear` $u = \min + x(\max - \min)$; `square` $u = \min + x^2(\max - \min)$ (finer near `min`); `log` $u = \min(\max / \min)^x$ (needs $\min > 0$)
observations (list; each entry is one of three kinds)		
`object`, `quantity`	–	a state or other quantity, e.g. `Storage`, `depth`, `"Age:concentration"`
`forcing`	–	a two-column file (time, value; comma separated); one input per window

Key	Default	Meaning
`windows`	[[0, 1]]	for `forcing`: list of `[a, b]` in control intervals; the input is the mean over $[t + a\Delta t,\ t + b\Delta t]$; windows with $b > 0$ are forecasts
`noise`	0	for `forcing`: log-normal forecast error (standard deviation of the log) applied to forecast windows
`season`	–	period (e.g. 365.25); adds sin and cos of the phase
`scale`	1	divide by this (aim for values of order one)
`transform`	`none`	`none` or `log1p` (after scaling; for skewed inputs such as flows)
objectives (list)		
`name`	required	an objective function in the script (method `integrate`)
`weight`	1	multiplies OHQ's own weight
`scale`	1	divides the objective
optional		
`initial_state`	[]	{`object, quantity, uniform: [lo, hi]`}: random initial state for training episodes
`snap`	[]	{`object, quantity, below`}: before each interval, a value with $\lvert x\rvert <$ `below` is set to 0
`baseline`	{}	reference fixed controls {`name: value`} (first row of every evaluation)
`solver_failure_penalty`	10	reward when the OHQ solve fails (the episode then ends)
training (all optional)		
`total_steps`	1 000 000	control intervals simulated in total (all workers)
`workers`	8	parallel OHQ models (CPU cores)
`rollout_steps`	256	intervals per worker between updates
`gamma`	0.99	discount per interval; the policy looks about $1/(1-\gamma)$ intervals ahead
`lam`	0.95	GAE parameter
`lr`, `epochs`, `minibatch`	3e-4, 10, 256	optimizer settings
`clip`, `vf_coef`, `ent_coef`	0.2, 0.5, 0	PPO settings
`normalize_rewards`	true	scale rewards by the running standard deviation of the return
`reward_clip`	0	clip normalized rewards (0: off)
`save_every`	25	checkpoint every N updates
`seed`	0	random seed
`warm_start`	none	start from a simple rule (Section 5.5)

5 Running

All commands are run from the OpenHydroQual folder.

5.1 Step 1: baselines

```
python -m ohqrl.evaluate model.yaml --windows validation \
    --constant "valve=0.05" "valve=0.1" "valve=0.2" --normalize --log baselines.json
```

Every evaluation runs the `baseline` controls first, then the policies you list. The `--constant` option takes comma-separated assignments for several controls, e.g. `"gate=0.3,pump=50"`. Each

policy is run on every window, without randomness. The script prints:
`J` the weighted objective, $\sum_i w_i \omega_i I_i / s_i$, averaged over the windows. Lower is better.
objective values the integral of each objective function over the window, i.e. OHQ's objective value.
`normalized` each objective divided by the baseline's (with `--normalize`).
Use the validation windows to choose among fixed settings. The test windows are kept for the end.

5.2 Step 2: training

```
python -m ohqrl.train model.yaml --out runs/model1
python -m ohqrl.train model.yaml --out runs/model2 --total-steps 500000 --workers 16
```

Command-line options override the `training` section. The run folder receives `config.yaml`, `train_log.txt`, the checkpoints `checkpoint_uNNNN.pt` and `final.pt`. Each line of the log is one update:

`intervals`	control intervals simulated so far
`episodes`	complete episodes so far
`mean episode return`	average total reward of the last 20 episodes (higher is better). It is measured *with exploration noise* and on random windows, so it is noisy and does not by itself show that the policy improved
`exploration std`	spread of the random exploration around the policy's action (in action units, $[-1, 1]$)
`intervals/s`	simulation speed

Run time. Roughly total_steps/(workers × intervals per second per worker). The pond example runs at about 30–40 intervals per second per worker.

5.3 Step 3: choose a checkpoint

```
python -m ohqrl.evaluate model.yaml --windows validation \
    --ppo "runs/model1/checkpoint_u*.pt" --select runs/model1/best.pt --log runs/model1/
        selection.json
```

Every checkpoint is run without exploration noise on the validation windows. The one with the lowest `J` is copied to `best.pt`. Choose this way rather than taking the last checkpoint. RL training is noisy, and later is not always better. Checkpoint `u0000` is the starting policy (the warm-start rule), so the selected policy is never worse than that rule on the validation windows.

5.4 Step 4: test

```
python -m ohqrl.evaluate model.yaml --windows test \
    --constant "valve=0.05" --ppo runs/model1/best.pt --normalize --log runs/model1/test.
        json
```

Compare the trained policy with the best fixed setting on windows that played no role in training or selection.

5.5 Warm start

Starting PPO from a reasonable rule saves most of the training time. The actor network is first fitted to the rule's actions (behavior cloning), then improved. Two forms are available:

```
training:
  warm_start: {controls: {valve: 0.1}} # fixed controls
  # or: warm_start: {rule: "mymodule:my_rule", kwargs: {threshold: 0.5}}
  # optional: steps (per worker, 2000), epochs (30), log_std (-1.6)
```

A rule is a Python function `my_rule(env, **kwargs)` that returns the action vector, with one entry in $[-1, 1]$ per control. Inside the rule, `env.sys` is the live OHQ model (e.g. `env.sys.get_value("Pond", "Storage")`) and `env.t` the current time. `env.action_from_control(c, u)` converts a control value to an action. `log_std` sets the exploration spread after cloning, as the natural logarithm of the standard deviation in action units (-1.6 means 0.2).

6 Worked example: stormwater pond

`ohqrl/examples/pond` contains a stormwater pond with a controllable outlet valve:

- **Inflow:** hourly runoff from a 2-ha impervious catchment, using the 1979–2025 rain record of Beltsville, MD (NOAA station USC00180700).
- **Layout:** the valve and an overflow weir discharge into a catch basin, which drains through a pipe to the outlet.
- **Water age:** a Laplace age tracer measures how "young" the outflow is.

There are two objectives:

- `Flood`: the integral of the squared outlet-flow excess above 1000 m^3/d;
- `Young`: the integral of outflow times tracer concentration, i.e. the young-water load.

Both are scaled by their values per year with the valve fully open, so the passive pond scores 1 per objective per year.

```
python ohqrl/examples/pond/make_inputs.py
python -m ohqrl.evaluate ohqrl/examples/pond/pond.yaml --windows test \
    --constant "valve=0.05" "valve=0.1" "valve=0.2" --normalize
```

Fixed opening	J (test, 2 windows of 3 yr)	Flood / passive	Young / passive
100% (passive)	8.24	1.000	1.000
20%	3.33	0.039	0.926
10%	2.80	0.005	0.820
5%	2.35	0.029	0.655

A small fixed opening already removes almost all of the flood objective and a third of the young-water load. A trained policy has to beat 2.35 to be worth having. It could do so, for example, by keeping the valve nearly closed in dry weather (aging the water) and opening it before forecast storms. Training follows with `python -m ohqrl.train ohqrl/examples/pond/pond.yaml --out runs/pond`, then Steps 3–4.

7 Using the environment from Python

`OHQControlEnv` is a standard Gymnasium environment, so it can be used with any RL library or with your own optimization loop:

```
from ohqrl.env import OHQControlEnv, constant_policy
env = OHQControlEnv("ohqrl/examples/pond/pond.yaml")
obs, info = env.reset(seed=1, options={"start": 43829, "initial_state": {}})
done = False
while not done:
    action = constant_policy(env, {"valve": 0.05}) # or your policy(obs)
    obs, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated
print(env.objective_totals()) # OHQ objective values for the window
env.close()
```

`reset` options:

- `start`: a fixed start time; otherwise a random training start.
- `initial_state`: {`(object, quantity): value`}, which replaces the random initial state. An empty dictionary keeps the script's values.

`info` contains the time, the control values applied, and the objective increments of the interval.

7.1 The OHQ stepping API

The environment rests on these methods of `openhydroqual_py.System`, which are also usable directly:

`begin_stepping(apply_parameters)`	initialize the solver as `solve()` does, without running
`advance_to(t)`	continue the run to time t without re-initializing; `False` if the solve failed
`end_stepping(uniformize_output)`	finalize the outputs
`current_time()`, `start_time()`, `end_time()`	simulation times
`get_value(obj, q)`, `get_values([(obj, q), ...])`	current value(s) of quantities
`set_value(obj, q, v)`	set a quantity now (present and past values)
`set_timeseries(obj, prop, t, v)`	replace a time-series property
`set_block_property`, `set_link_property`	set a property by its text value
`objective_names()`, `objective_method(n)`, `objective_weight(n)`	objective functions in the model
`objective_integral(n)`	integral of objective n from the start of the run to now
`objective_value(n)`	the objective as OHQ evaluates it (its method)

8 Practical advice

8.1 Speed

- The OHQ solve is almost all of the cost. Use the largest solver step (`initial_time_step`) that is accurate, and `max_timestep_increase_factor: 1` if the step should not grow.
- Near-empty storages left at round-off levels (around 10^{-9}) can slow the solver by orders of magnitude. List them under `snap`. In the pond example, a 30-day test that took 94 s (and then failed) without snapping ran in 3 s with it.

- A long control interval means fewer decisions per episode and faster training, at the cost of a less responsive policy.
- Set `make_timeseries_uniform: no` (already in the example script) when forcing files are long: resampling a multi-decade record at every episode start is slow.

8.2 Long runs

An episode is one long OHQ run. `ohqrl` raises OHQ's whole-run limits on matrix inversions (`maximum_number_of_matrix_inverstions`, so spelled in OHQ) and wall-clock time (`maximum_time_allowed`). Without this, multi-year episodes fail partway. Both can be overridden under `model: settings`.

8.3 Getting a policy that is really better

These lessons come from the reservoir, pond and environmental-flow studies in the OpenHydroQual-RL project:

1. **Tune the simple rules first.** A grid of fixed settings, or a simple threshold rule, is often hard to beat. Report the trained policy against the *best* of them.
2. **Train on what you evaluate.** `ohqrl` guarantees this: the reward sums to the objective. If you add terms of your own, keep them consistent with how you will judge the result. Otherwise RL optimizes the difference.
3. **Judge without noise.** The training curve includes exploration noise. Always select and test with `ohqrl.evaluate`, which runs the policy deterministically.
4. **Exploration noise can bias the result.** If small, random changes of the control are themselves penalized, the noisy training policy is worse than the policy it is learning, and training drifts. Examples are thresholds that a slightly lower release misses, and limits on day-to-day changes. Use smooth objectives, a smaller `log_std` after the warm start, or a longer control interval.
5. **Look ahead far enough.** The policy weighs rewards about $1/(1-\gamma)$ intervals ahead. Make that horizon cover the effect of a decision, e.g. a storm event for a pond, or a season for a reservoir. `gamma` close to 1 trains more slowly.
6. **Keep test windows out of every decision**, including the choice of scales, rule settings and checkpoints.

8.4 Troubleshooting

Symptom	Cause and remedy
`openhydroqual_py not found`	bindings not built, or in an unexpected folder: set `OHQ_PY_PATH`
`OHQ could not build the model`	script error. Load it in the OHQ GUI or run it with the terminal version for messages. Objective functions need `type=Objective_Function`

Symptom	Cause and remedy
`... uses method 'maximum'`	only `integrate` objectives are accepted: rewrite as an integral (Section 3.4)
`KeyError: not found: X:Y`	block/link `X` has no quantity `Y`; check names and capitalization (`Storage`, `depth`, `"Age:concentration"`)
episodes end early with `solver_failed`	the OHQ solve failed: smaller `initial_time_step`, `snap` near-zero states, check the model alone over the same window
very slow intervals	round-off states (use `snap`), very small solver steps, or many workers on few cores
control has no effect	the property is not used by the flow expression after initialization, or the wrong `kind`
`no training range is longer than the episode`	`train` ranges shorter than `episode_length`

8.5 Limitations

- Objectives must be additive (time integrals).
- The policy sees only what is listed under `observations`. Hidden states that matter should be included.
- The bundled trainer is PPO with small fully connected networks. Other algorithms can use `OHQControlEnv` directly.
- Each worker rebuilds the model from the script at every episode start. This is cheap compared with an episode, but not free for very large models.